

Camilo A. Cuellar
Deep Learning, 2026-2
Universidad de Antioquia

Contexto de aplicación

Pronosticar demanda en un supermercado no consiste solo en acertar el total vendido. Por ejemplo, si se esperan pocas bebidas en una tienda durante un fin de semana con promoción, puede faltar producto aun cuando el pronóstico global sea razonable. El error contrario también cuesta: comprar demasiado inmoviliza inventario y puede aumentar el desperdicio de productos perecederos. En la competencia de Kaggle se plantea precisamente estas dos situaciones (exceso de inventario y faltantes) como motivación de la solución [1].

Conviene aclarar varios términos antes de seguir: se llama “familia” a un tipo amplio de producto (como BEVERAGES o PRODUCE) y cada combinación de tienda y familia forma una serie diaria. Además, la variable “onpromotion” indica cuántos artículos de una familia específica estaban en promoción en una tienda y fecha determinadas. Por último, en el reporte se usará la palabra ventas para el target “sales”.

Objetivo de machine learning

La pregunta es: **¿cuántas ventas tendrá cada familia en cada tienda durante las 16 fechas siguientes al origen del pronóstico?** La unidad predicha queda definida por tienda, familia y fecha. Se trata de un forecast directo de varios horizontes, coherente con las 16 fechas reales de test.

Para construir cada predicción se usan ventas y transacciones observadas hasta el origen, metadatos de la tienda y variables de calendario. Las variables futuras se tratan según su disponibilidad real al momento de pronosticar. Las promociones se incluyen porque son decisiones que el comercio podría planear. Los festivos también se conocen de antemano, aunque habría que confirmar si son nacionales, regionales o locales. El precio del petróleo se usa ya que es incluido en el periodo de test: es una decisión de este experimento para reproducir la interfaz de la competencia, no una suposición trasladable a un despliegue real, donde habría que pronosticarlo. Las transacciones, en cambio, solo aparecen hasta el final de entrenamiento, así que nunca se usan valores futuros.

Dataset: tipos de datos y distribución

El dataset contiene siete CSV que ocupan 124,7MB. Train tiene 3.000.888 filas entre el 1 de enero de 2013 y el 15 de agosto de 2017. Test tiene 28.512 filas entre el 16 y el 31 de agosto de 2017. El panel completo combina 54 tiendas con 33 familias, para un total de 1.782 series tienda-familia. Los demás archivos describen tiendas, transacciones, petróleo, festivos y el formato de entrega.

Variable o grupo	Tipo original	Qué representa	Disponibilidad y uso
date	datetime64	Fecha de la observación o del pronóstico	Conocida; de ella salen variables calendario.
store_nbr	int64, tratado como categórico	Tienda	Conocida; también enlaza los metadatos y transacciones.
family	string, categórica	Tipo amplio de producto	Conocida; junto con la tienda identifica cada serie.
sales	float64, continua	Ventas diarias de una tienda-familia	Es el target; solo se usan sus valores históricos.
onpromotion	int64, conteo	Artículos de la familia que están en promoción	Disponible en train y test; se puede conocer por la planeación comercial.
city, state, type, cluster	string e int64, categóricas	Ubicación y agrupación de la tienda	Metadatos estáticos conocidos antes del forecast.
transactions	int64, conteo	Transacciones diarias de una tienda	Solo se usan lags y ventanas históricas; no existe en el horizonte de test.
dcoilwtico	float64, continua	Precio diario del petróleo	Kaggle lo entrega para train y test; fuera de la competencia debe auditarse.

Variables de festivos	fecha, categóricas y booleana	Tipo, alcance, descripción y traslado del evento	El calendario es conocido, pero debe resolverse su alcance por tienda.
-----------------------	-------------------------------	--	--

En lugar de una distribución de clases se analizaron ceros, cuantiles y segmentos, porque ventas es un target continuo no negativo. Los valores fraccionarios son válidos: Kaggle explica que algunos productos pueden venderse por peso, por ejemplo una cantidad de queso en kilogramos. La distribución se puede leer en tres partes:

- Ceros e intermitencia: 31,30 % de las filas tiene ventas iguales a cero. El 15,40 % contiene ventas fraccionarias, lo cual confirma que es un problema de regresión y no de conteo entero.
- Centro: la mediana es 11 y el percentil 90 es 867. La mayoría queda lejos de los valores más altos.
- Cola: los percentiles 95 y 99 son 1.965 y 5.507; el máximo llega a 124.717. No se ha considerado eliminar estos outliers sin antes comprobar que puedan representar una mejora.

Métricas y protocolo temporal

La métrica principal es RMSLE, la usada oficialmente por la competencia [1]:

$$RMSLE = \sqrt{\frac{1}{n} \sum_{i=1}^n (\log(\hat{y}_i + 1) - \log(y_i + 1))^2}$$

Un valor menor es mejor. El logaritmo evita que las series de mayor volumen dominen por completo la evaluación y mantiene atención sobre errores relativos. La implementación local comprueba las formas y los valores de entrada; si un modelo produce una predicción negativa, se recorta a cero antes de evaluar. Además del resultado global, se revisa RMSLE por horizonte, tienda, familia, promoción y régimen temporal.

Como lectura de negocio se propone el sesgo porcentual agregado por tienda y por familia.

$$\text{Sesgo}(\%) = 100 \cdot \frac{\sum(\text{Predicción} - \text{venta})}{\sum \text{venta}}$$

Un valor positivo indica sobreestimación agregada y uno negativo, subestimación. Esta medida adapta el error firmado descrito en [2] a grupos útiles para inventario. No se traduce todavía a dinero, porque el dataset no incluye costos de mantener inventario, faltantes o desperdicio.

La evaluación usa rolling origin: se fija una fecha de corte, se entrena solo con lo anterior y se pronostican los 16 días siguientes. Después el corte avanza y se repite el ejercicio [2]. Se usaron tres folds, cada uno con 28.512 predicciones fuera de fold, para 85.536 por modelo. La literatura sobre validación en series temporales respalda este esquema frente al k-fold aleatorio cuando hay dependencia temporal [3]

Dentro de cada fold, los lags y ventanas móviles terminan en el origen. La imputación y los vocabularios categóricos se ajustan únicamente con el tramo de entrenamiento. Los festivos se reducen a una fila por tienda y fecha para evitar que un join duplique ventas. Un detalle que ocurrió al construir los folds es que el origen del 18 de diciembre de 2016 se tuvo que descartar en los tres folds porque su ventana objetivo incluye el 25 de diciembre, fecha que no existe en train.csv.

Resultados iniciales

El EDA cambió tres decisiones de diseño. La media diaria agregada de los últimos 90 días es 2,45 veces la de los primeros 90, así que asumir un nivel estable no era defendible y eso obligó al esquema rolling-origin con escalado ajustado por fold. La combinación de 31 % de ceros con cola larga obligó a la escala logarítmica. Y aunque promociones, transacciones, petróleo y festivos muestran asociaciones fuertes con ventas (la correlación de ventas agregadas con el petróleo es -0,705), esas relaciones están confundidas con el crecimiento de la demanda en el mismo periodo, así que quedaron como candidatas a validar por ablación y no como hallazgos.

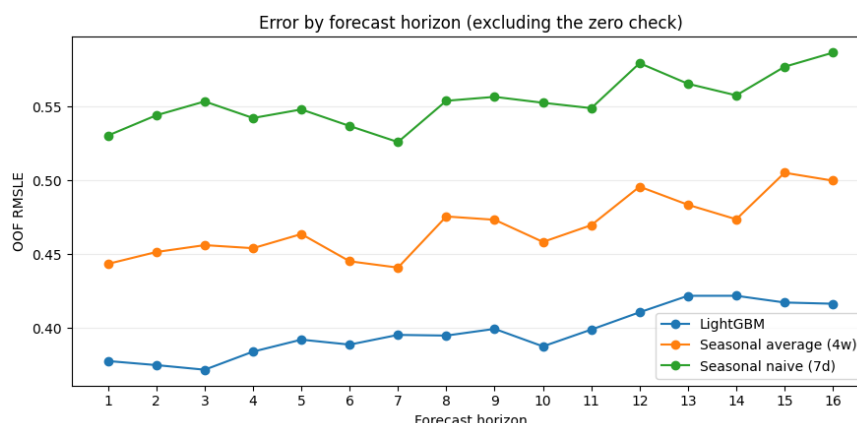
El pipeline de features produjo 73 variables, 67 numéricas y 6 categóricas, calculadas con marcas de tiempo estrictas por serie, y pasó las pruebas negativas de fuga antes de entrenar cualquier modelo.

Los cuatro métodos se evaluaron sobre exactamente los mismos folds. La predicción cero es un control de sanidad; el seasonal naive repite el valor de una semana atrás; la variante estacional promedia el mismo día de la semana en las últimas cuatro semanas; LightGBM [4] se eligió antes de entrenar como el único modelo de árboles, para no escoger entre familias después de ver resultados.

Modelo	Global	Fold 1	Fold 2	Fold 3
Cero, control	4,4197	4,4202	4,4193	4,4195
Seasonal naive semanal	0,5539	0,5176	0,5212	0,6170
Promedio estacional de cuatro semanas	0,4684	0,4245	0,4427	0,5311

LightGBM global	0,3973	0,3782	0,3981	0,4148
-----------------	--------	--------	--------	--------

LightGBM gana en global y en los tres folds, con una mejora del 15 % sobre el promedio estacional. Ese resultado es consistente con lo reportado en competencias recientes de forecasting, donde los modelos de árboles entrenados sobre muchas series a la vez superan a los métodos univariados clásicos [5]. Aun así, el promedio estacional se queda como comparador porque cuesta 5,7 segundos de cómputo contra 178,5 de LightGBM.



Se espera que el error crezca de forma limpia con la distancia al origen, y sí crece en términos generales, de 0,3717 en el día 3 a 0,4217 en el día 14, un aumento cercano al 13 %. Pero la curva no es monótona: hay días tardíos que quedan alrededor de 0,41-0,42 sin seguir un orden estricto, y algunos días intermedios que mejoran respecto al anterior. Esto tiene una lectura importante para el diseño: al avanzar el horizonte no solo aumenta la distancia temporal, también cambia el día de la semana y la posición dentro del mes. Por eso no se puede atribuir el deterioro únicamente a la distancia del pronóstico.

Los cortes por segmento apuntan a dónde mirar después. Las filas sin promoción tienen un RMSLE de 0,4440 frente a 0,3252 de las promocionadas, lo cual es contraintuitivo si uno espera que la promoción introduzca volatilidad; una explicación plausible es que las filas promocionadas se concentran en series de alto volumen y comportamiento más regular, pero eso hay que verificarlo controlando por tienda y familia. El fold 3 fue el más difícil para los tres métodos no triviales, sin que por ahora haya evidencia para atribuirlo a una tienda, familia o evento en particular.

El siguiente paso es entrenar una red recurrente sobre estos mismos tres folds, bajo el argumento de que una arquitectura secuencial [6] puede representar patrones que las 73 features resumidas al origen no conservan, tal como se ha explorado en forecasting sobre paneles de muchas series [7]; para considerarla una mejora real quedó pre-registrado un umbral de RMSLE global $\leq 0,3894$, es decir, 2 % por debajo de LightGBM.

Referencias

- [1] Kaggle, Store Sales - Time Series Forecasting: Overview, Data y Evaluation. <https://www.kaggle.com/competitions/store-sales-time-series-forecasting>
- [2] R. J. Hyndman y G. Athanasopoulos, Forecasting: Principles and Practice, 3.^a ed., OTexts, 2021. Capítulos de validación cruzada en series temporales y evaluación de precisión. <https://otexts.com/fpp3/tscv.html>
- [3] C. Bergmeir y J. M. Benítez, "On the use of cross-validation for time series predictor evaluation", Information Sciences, vol. 191, pp. 192-213, 2012.
- [4] G. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [5] S. Makridakis, E. Spiliotis y V. Assimakopoulos, "M5 accuracy competition: Results, findings, and conclusions", International Journal of Forecasting, vol. 38, n.º 4, pp. 1346-1364, 2022.
- [6] S. Hochreiter y J. Schmidhuber, "Long Short-Term Memory", Neural Computation, vol. 9, n.º 8, pp. 1735-1780, 1997.
- [7] D. Salinas, V. Flunkert, J. Gasthaus y T. Januschowski, "DeepAR: Probabilistic forecasting with autoregressive recurrent networks", International Journal of Forecasting, vol. 36, n.º 3, pp. 1181-1191, 2020.
- [8] Resultados propios del proyecto: 01_data_audit_edu.ipynb, 02_feature_pipeline.ipynb y 03_non_neural_baselines.ipynb.